POCKETPAL

Student Pocket Money Management Application

A Mini Project Report

Submitted By

Name	Roll No.
Grishma Naik	2619
Laxita Gavde	2627
Noel Luis	2630

Under the Guidance of

Mr. Hanumant Redkar
Mr. Sarvesh Velip

Master of Computer Applications (MCA)

Goa University

Date: **Year: 2026–2027**

DECLARATION

We hereby declare that the mini-project entitled “PocketPal: Student Pocket Money Management Application” has been developed by us as part of the academic requirements for the degree of Master of Computer Applications (MCA) at Goa University, under the guidance of Mr. Hanumant Redkar and Mr. Sarvesh Velip.

This project is designed to help students manage their pocket money by tracking their daily expenses, planning their budget, setting savings goals, and developing better money-management habits.We declare that this project has been completed by our team as part of our academic work. The information presented in this report is based on our project development, implementation, and study.

Grishma (2619) **Laxita (**2627) **Noel** (2630)

Place: Goa University

Date: _____

PREFACE

The mini-project **“PocketPal: Student Pocket Money Management Application”** has been developed as part of the academic requirements for the Master of Computer Applications (MCA) programme at Goa University.

Pocket money is an important part of student life, and managing it properly can sometimes be difficult. Students often spend money on food, transportation, study materials, and other daily needs without keeping a proper record of their expenses. PocketPal is designed to help students manage their pocket money by recording their income and expenses, monitoring their spending, planning their budget, and setting savings goals.This project provided us with an opportunity to apply our knowledge of Java programming, object-oriented programming concepts, and Android application development. It also helped us understand the process of planning, designing, developing, and organizing a mobile application.

We would like to express our sincere gratitude to our guides, Mr. Hanumant Redkar and Mr. Sarvesh Velip, for their guidance and support throughout the development of this project. We are also thankful to Goa University for providing us with the opportunity to undertake this mini-project as part of our academic learning.Finally, we hope that this project demonstrates how a simple mobile application can help students develop better financial management habits and become more aware of their spending.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of our mini-project, “PocketPal – Student Pocket Money Management Application.”

We are especially thankful to our project guides, Mr. Hanumant Redkar and Mr. Sarvesh Velip, for their valuable guidance, encouragement, and suggestions throughout the project. Their support helped us understand the different stages of application development and improve our approach to designing and implementing the application.

We would also like to thank the Head of the Department and all the faculty members of Goa University for providing us with the necessary support and academic guidance during the completion of this project.

We are grateful to Goa University for giving us the opportunity to undertake this mini-project as part of our Master of Computer Applications (MCA) programme. This project helped us strengthen our knowledge of Java programming, object-oriented programming concepts, Android application development, user interface design, and data management.

Finally, we would like to thank our family and friends for their encouragement, patience, and support throughout the project.This mini-project has been a valuable learning experience for our team, allowing us to apply our theoretical knowledge to the development of a practical mobile application designed to help students manage their pocket money and develop better financial habits.

TABLE OF CONTENTS

Sr. No.	Contents
	Declaration
	Preface
	Acknowledgement
	Table of Contents
	Abstract
1	CHAPTER 1: INTRODUCTION
1.1	Aim and Objectives
1.2	Research Question
1.3	Scope of the Project
2	CHAPTER 2: METHODOLOGY
2.1	Understanding the Problem
2.2	Problem Statement
2.3	Requirement Analysis
2.3.1	Functional Requirements
2.3.2	Non-Functional Requirements
2.3.3	Hardware Requirements
2.3.4	Software Requirements
2.4	System Design
2.4.1	System Architecture
2.5	UML Diagrams and System Modelling
2.6	Object-Oriented Programming Concepts
3	CHAPTER 3: RESULT ANALYSIS
3.1	Splash Screen and Application Introduction
3.2	Home Screen — Dashboard
3.3	Income and Expense Management
3.4	Budget Management
3.5	Savings Goals
3.6	Transaction History
3.7	Spending Summary and Categories
3.8	User Profiles and Settings
4	CHAPTER 4: CONCLUSION AND FUTURE SCOPE
4.1	Conclusion
4.2	Future Scope
5	References

ABSTRACT

PocketPal is an Android mobile application developed to help students manage their pocket money effectively. Students often find it difficult to keep track of their daily expenses and manage their limited funds. Without proper expense records and budgeting, they may spend more than planned and find it difficult to save money. PocketPal aims to address this problem by providing a simple and user-friendly platform for managing personal finances.

The application allows users to record their income and expenses, monitor their spending, manage their monthly budget, and set savings goals. It also provides a dashboard to view financial summaries and a spending breakdown to understand how money is spent across different categories. The application supports multiple user profiles and stores financial information locally using a Room database, with a JSON file used for backup and recovery.

PocketPal is developed using Java and Android development tools. The project applies object-oriented programming concepts to organize the application into separate classes and components. It also provides practical experience in mobile application development, user interface design, data management, and software development.

The main objective of PocketPal is to encourage students to develop responsible spending habits, plan their pocket money, and improve their saving practices. The application serves as a practical tool for making students more aware of their financial activities and helping them manage their money in a more organized way.

CHAPTER 1 : INTRODUCTION

Managing pocket money is an important part of student life. Students receive money from their parents, guardians, scholarships, part-time work, or other sources to meet their daily needs. They spend this money on various activities such as food, transportation, study materials, mobile recharges, shopping, and entertainment. However, many students do not maintain a proper record of their spending. As a result, they may find it difficult to understand where their money goes, control unnecessary expenses, and save money for future needs.

With the increasing use of smartphones, mobile applications can provide a simple way to manage personal finances. An expense management application allows users to record their income and expenses, monitor their spending, and plan their budget. Such applications can help students become more aware of their financial habits and make better decisions about how they use their pocket money.

PocketPal - Student Pocket Money Management Application is an Android application developed especially for students to help them manage their pocket money in an organized manner. The application provides features for recording income and expenses, managing a monthly budget, setting savings goals, and viewing spending summaries. It also allows users to check their transaction history and understand their spending across different categories.

The application is developed using Java and Android development tools. It uses a Room database to store financial records locally and a JSON file to support backup and recovery. The application is organized into different classes and components, applying object-oriented programming concepts to make the code easier to understand and maintain.

The main purpose of PocketPal is to encourage students to develop good financial habits from an early stage. By keeping track of their spending and setting savings targets, students can learn to use their pocket money more responsibly. The application aims to provide a simple and convenient platform that supports better budgeting, spending awareness, and saving practices.

1.2 Aim and Objectives

Aim

The main aim of PocketPal is to develop an Android application that helps students manage their pocket money effectively by tracking expenses, planning budgets, and encouraging savings.

Objectives

1. To provide a simple platform for students to record and monitor their daily income and expenses.
2. To help students manage their pocket money through monthly budgeting, spending summaries, and savings goals.
3. To encourage responsible spending habits and improve students' awareness of their financial activities.

1.3 Research Question

How can an Android application help students manage their pocket money, track their daily expenses, and develop better budgeting and saving habits?

1.4 Scope of the Project

The scope of PocketPal is limited to personal pocket money management for students. The application provides features to record income and expenses, view transaction history, organize expenses into categories, monitor monthly budgets, and set savings goals. It also displays financial summaries and category-wise spending information to help students understand their spending patterns.

The application supports multiple user profiles and stores data locally on the device using a Room database, with JSON-based backup and recovery functionality. It is developed as an Android application using Java and is intended for individual financial tracking and basic budgeting.

PocketPal focuses on helping students maintain better control over their available pocket money and develop regular saving habits. It does not function as a banking application and does not directly connect to bank accounts or process financial transactions. Its purpose is to record and organize financial information entered by the user and present it in an understandable format.

CHAPTER 2 : METHODOLOGY

2.1 Understanding the Problem

Managing pocket money is an important part of student life. Students receive money from their parents, guardians, scholarships, part-time work, or other sources. They use this money for food, transportation, stationery, mobile recharges, educational materials, entertainment, and other daily needs.

However, students may find it difficult to keep track of how much money they receive and how much they spend. When expenses are recorded mentally or on paper, it becomes difficult to calculate total spending, identify unnecessary expenses, and understand where the money is being used. Students may also find it challenging to save money for future needs because they do not always have a clear picture of their available balance.

PocketPal was developed to address these problems by providing a simple mobile application for managing student pocket money. The application allows users to record income and expenses, monitor their spending, manage a budget, and set savings goals. It also provides information about recorded transactions so that users can review their financial activities.The main idea behind PocketPal is to make pocket money management easier, more organised, and accessible through a mobile application. The application aims to encourage students to develop better spending habits and become more aware of their financial activities.

2.2 Problem Statement

Students often face difficulties in managing their pocket money because they may not maintain proper records of their income and daily expenses. Manual methods of recording expenses can be inconvenient and may not provide a clear understanding of spending patterns, available balances, and savings progress.

Therefore, there is a need for a simple and user-friendly mobile application that helps students record their income and expenses, monitor their budget, review their transaction history, and manage savings goals in one place.PocketPal aims to provide this solution through an Android application developed using Java. It is intended to help students organise their pocket money records and make better financial decisions.

2.3 Requirement Analysis

Requirement analysis is the process of identifying the functions and resources required to develop and operate a software application. It helps define what the system should do and the conditions under which it should work.

For PocketPal, the requirements were identified according to the needs of students who want to manage their pocket money and keep track of their spending.

2.3.1 Functional Requirements

Functional requirements describe the main operations that the application should provide to its users.

The functional requirements of PocketPal are as follows:

1. **Income Management:** The application provides functionality for recording income received by the user, including the amount, category, and other relevant transaction details.
2. **Expense Management:** Users can record their expenses to maintain a history of their spending.
3. **Transaction History:** The application stores transaction details so that users can review their previously recorded income and expenses.
4. **Budget Management:** The application provides budget-related functionality to help users plan and monitor their spending.
5. **Savings Goals:** Users can create and manage savings goals to keep track of their intended savings.
6. **Dashboard:** The application provides a dashboard to display important financial information in an organised manner.
7. **Expense Categories:** Transactions can be organised into categories to help users understand how their money is being spent.
8. **User Profiles:** The application includes profile-related functionality to organise financial information according to the selected profile.
9. **Local Data Storage:** The application stores relevant data locally so that recorded information can be accessed by the application without requiring a remote server.
10. **Settings and Preferences:** The application provides settings-related functionality for managing supported application preferences.

2.3.2 Non-Functional Requirements

Non-functional requirements describe the quality, usability, and operational characteristics expected from the application.

1. **Usability:** The application should have a simple and understandable interface that students can use without difficulty.
2. **Performance:** Common operations, such as viewing the dashboard and recording transactions, should be performed efficiently.
3. **Reliability:** The application should handle recorded financial information consistently and reduce the possibility of accidental loss during normal use.
4. **Maintainability:** The source code should be organised into classes and components so that the application can be understood, corrected, and updated more easily.
5. **Data Management:** The application should maintain transaction and related information in an organised local storage system.
6. **Compatibility:** The application should be developed for compatible Android devices and operating-system versions.

7. **Readability:** The interface should present financial information clearly so that users can understand their recorded transactions and budget details.

2.3.3 Hardware Requirements

The following hardware is suitable for developing and running the PocketPal application.

- A computer or laptop capable of running Android Studio.
- A processor, RAM, and storage capacity sufficient for Android development tools.
- An internet connection for downloading development tools, SDK components, and required dependencies.
- An Android device or a compatible Android emulator for running the application.

The exact hardware requirements depend on the Android Studio version, emulator configuration, and Android SDK components installed.

2.3.4 Software Requirements

The main software and development tools used for the project are listed below.

- **Programming Language:** Java
- **Development Environment:** Android Studio
- **Platform:** Android
- **Build System:** Gradle
- **Local Database:** Room persistence library
- **Additional Local Storage:** JSON-based storage for backup and recovery functionality
- **User Interface:** Android View-based components

These tools support the development of the Android application, implementation of its logic, management of local data, and organisation of the project source code.

2.4 System Design

System design describes the structure of the application and explains how its components interact to provide the required functionality. It serves as a plan for implementing the system before and during coding.

The design of PocketPal includes the application structure, user interactions, Java classes, object relationships, execution sequences, and data storage components. UML diagrams are used to represent different aspects of the system.

2.4.1 System Architecture

PocketPal follows an organised application structure in which the user interface, application logic, data-handling components, and data storage have separate responsibilities.

The main components are described below.

1. User Interface Layer

The user interface layer contains the screens and visual components through which users interact with PocketPal. It presents information and allows users to perform supported operations, such as recording transactions and viewing financial details.

2. Application Logic Layer

The application logic layer manages operations required by the application. The PocketPalViewModel class coordinates relevant application state and operations, including navigation and transaction-related functionality.

3. Repository Layer

The PocketPalRepository class acts as an intermediary between the application logic and the data storage components. It coordinates data operations and supports communication with the local database and JSON-based storage functionality.

4. Database Layer

The database layer uses the Room persistence library to manage structured local data. The application includes database entities and Data Access Object (DAO) interfaces for performing supported database operations.

5. JSON Storage Component

The JsonStorageManager class supports JSON-based local storage used for backup and recovery functionality. It provides an additional mechanism for handling application data alongside the primary Room database.

These components work together to support the application's operations. Separating responsibilities across classes helps organise the source code and makes the application easier to understand and maintain.

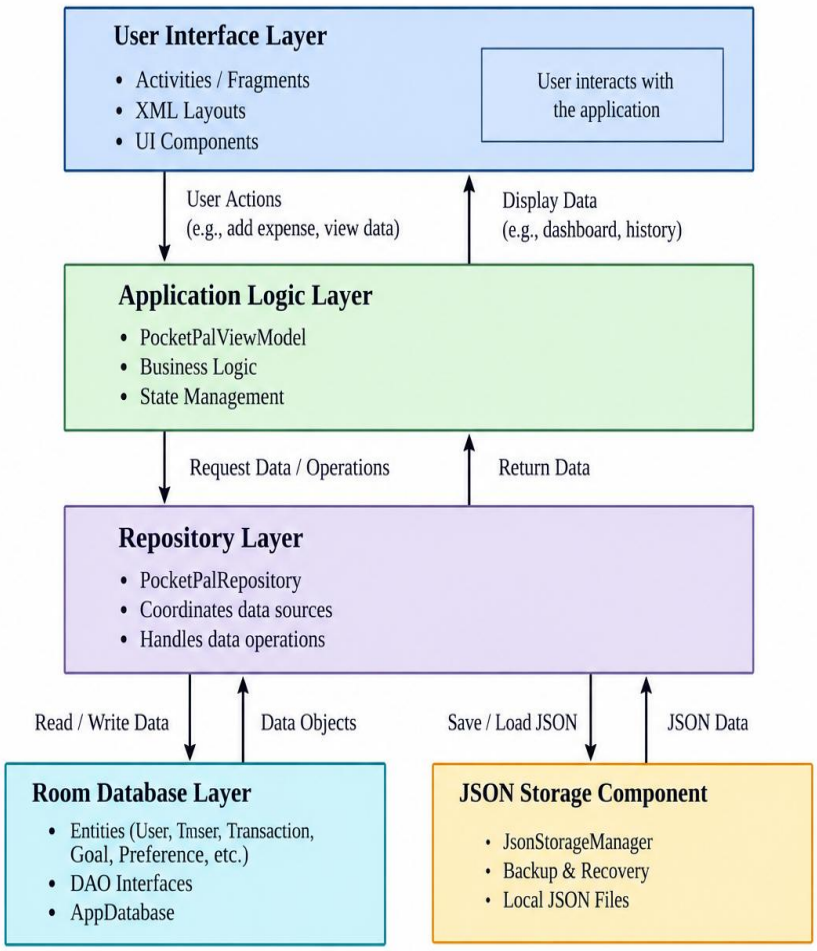


Figure 2.1: System Architecture of PocketPal

Figure 2.1: System Architecture of PocketPal

2.4.2 Use Case Diagram

A use case diagram represents the interactions between an external actor and the functions provided by a system. It gives an overview of what a user can do with the application without describing the internal implementation.For PocketPal, the primary actor is the user. The user interacts with the application to manage pocket money and review financial information.

The use cases may include:

- Accessing the application.
- Viewing the dashboard.
- Recording income.
- Recording expenses.
- Viewing transaction history.
- Managing budget information.
- Creating or managing savings goals.
- Reviewing expense categories.
- Managing supported profiles and preferences.

Only include use cases that are represented in your actual diagram and supported by the application.

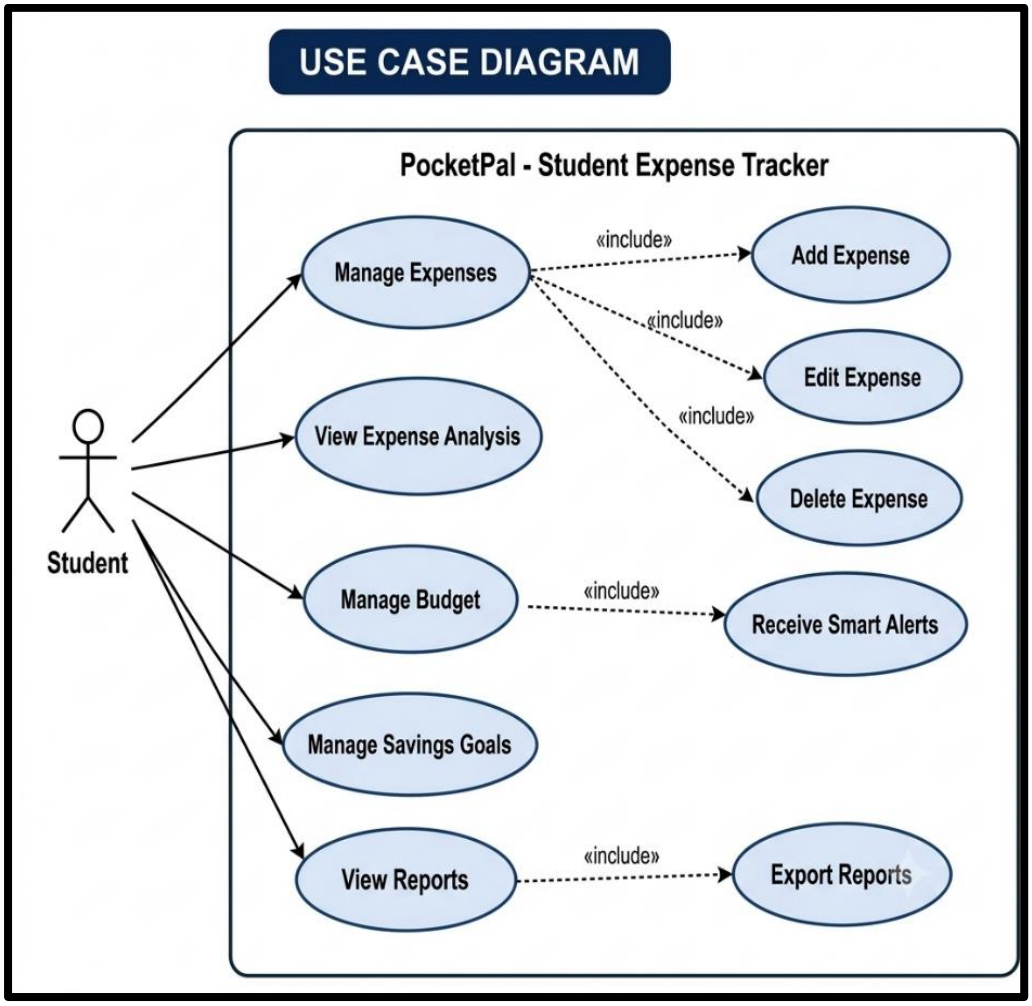


Figure 2.2: Use Case Diagram of PocketPal

2.4.3 Class Diagram

A class diagram represents the static structure of a system. It shows the classes, their attributes, methods, and relationships with other classes.PocketPal is developed using Java, so classes are used to organise its data and operations. Examples of important classes include MainActivity, PocketPalViewModel, PocketPalRepository, AppDatabase, data model classes, and supporting UI components.

The classes have different responsibilities. For example, the activity provides an entry point for the user interface, the ViewModel coordinates application operations, the repository handles data-related operations, and model classes represent information used by the application.

The class diagram also helps illustrate object-oriented programming concepts such as inheritance, association, and other relationships between classes.

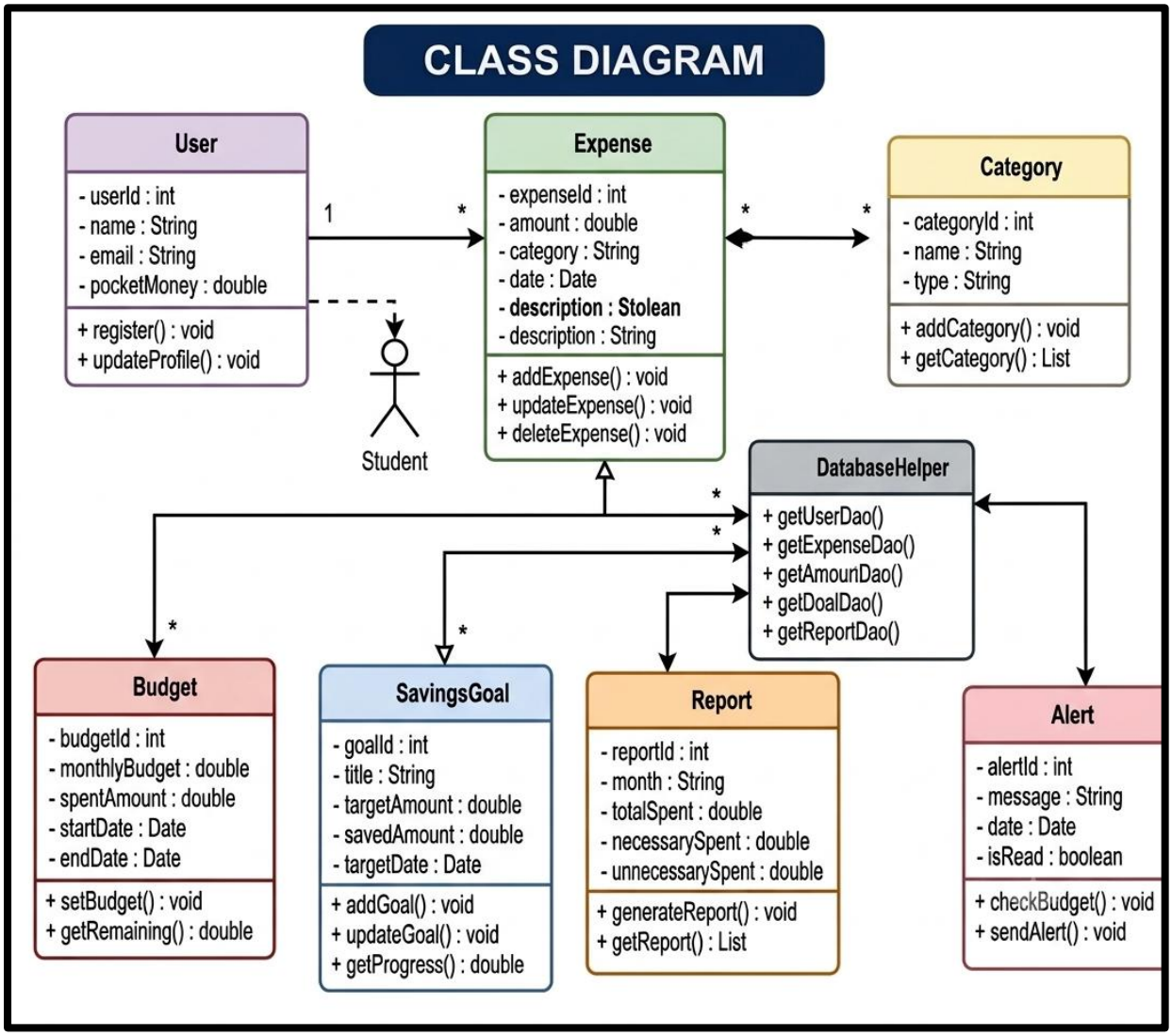


Figure 2.3: Class Diagram of PocketPal

2.4.4 Object Diagram

An object diagram represents a snapshot of objects and their relationships at a particular point in time. Unlike a class diagram, which describes the general structure of classes, an object diagram shows specific instances of those classes.

In PocketPal, objects are created from classes to perform operations and represent information. For example, an object created from a transaction model can represent a particular income or expense record. Objects may also reference other objects when working together to perform application operations.

The object diagram helps explain how instances of classes may be related during the execution of the application.

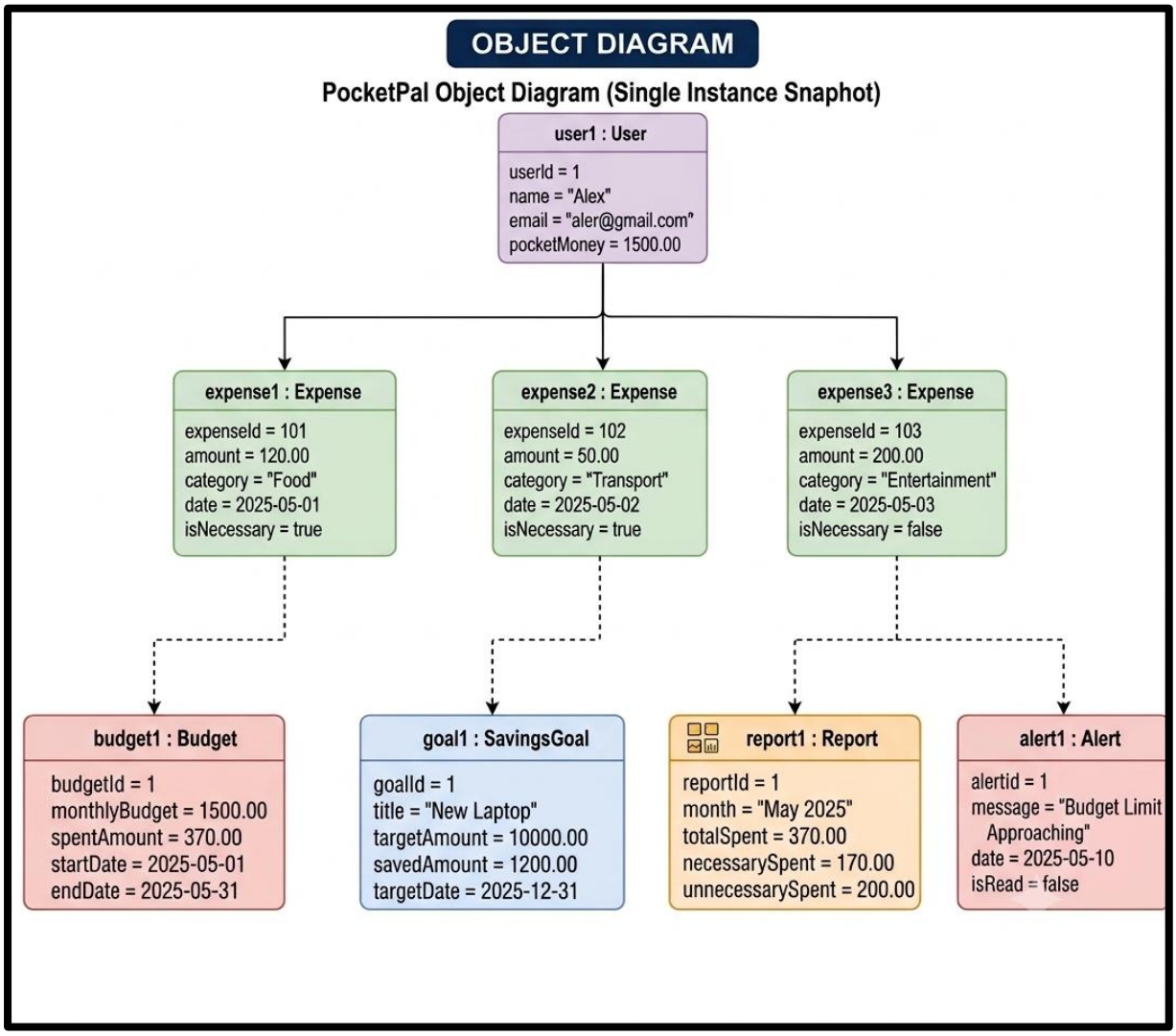


Figure 2.4: Object Diagram of PocketPal

2.4.5 Sequence Diagram

A sequence diagram represents the order in which messages are exchanged between objects during a particular operation. It focuses on the time sequence of interactions required to complete a task.

For example, when a user performs a transaction-related operation, the user interface receives the input, the relevant application logic processes the operation, and the repository coordinates the required data handling. The database component is involved when data needs to be stored or retrieved.

A sequence diagram helps explain the communication between the components and the order in which the operations occur.

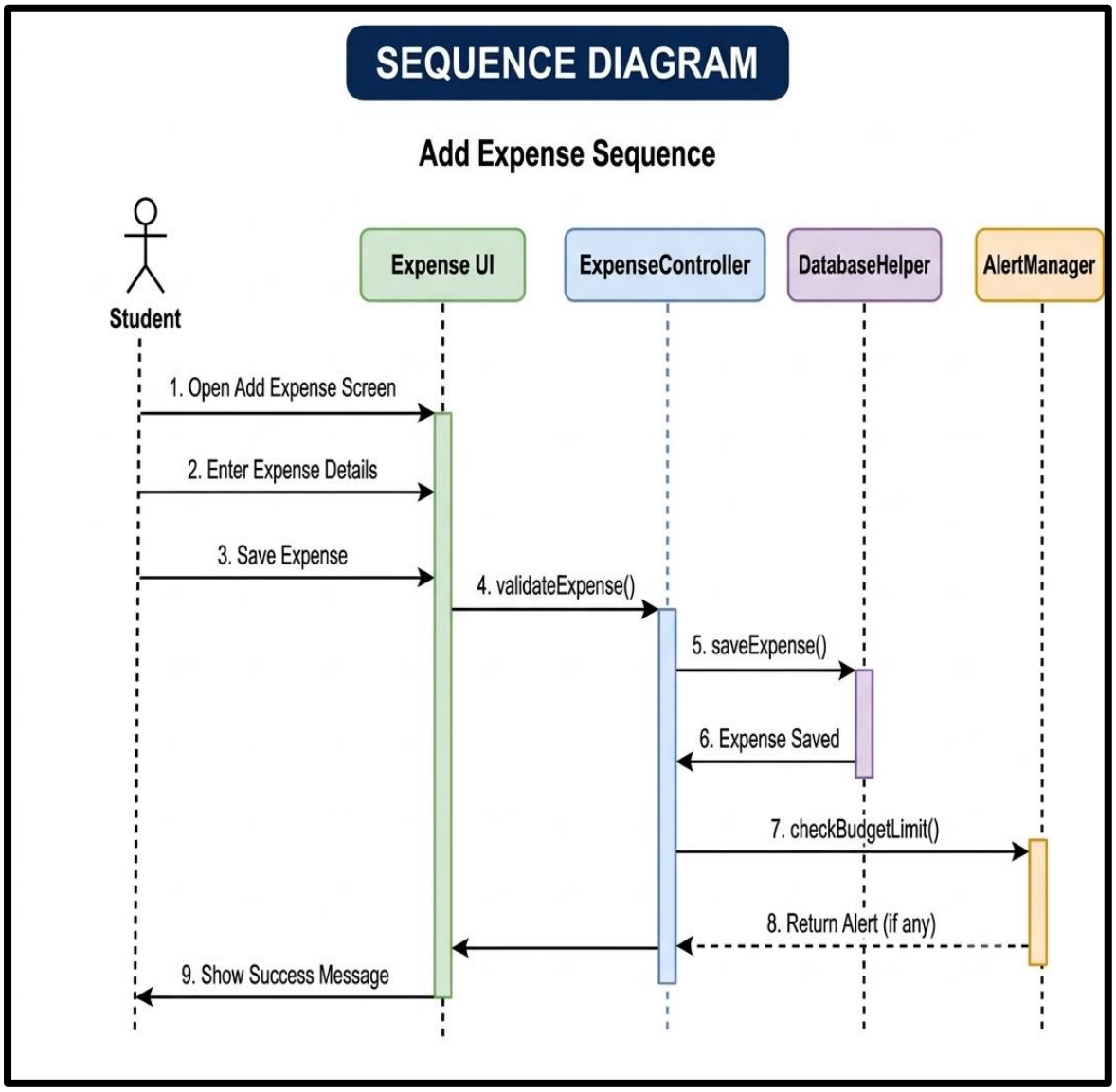


Figure 2.5: Sequence Diagram of PocketPal

2.4.6 Activity Diagram

An activity diagram represents the workflow of an operation. It shows the sequence of activities, decisions, and possible paths followed while completing a task.

In PocketPal, activity diagrams can be used to describe workflows such as recording an expense, entering transaction details, saving information, and viewing the updated financial information.

The diagram helps explain the steps followed by the user and the system during an operation. Decision points can be used to represent situations in which the workflow may follow different paths.

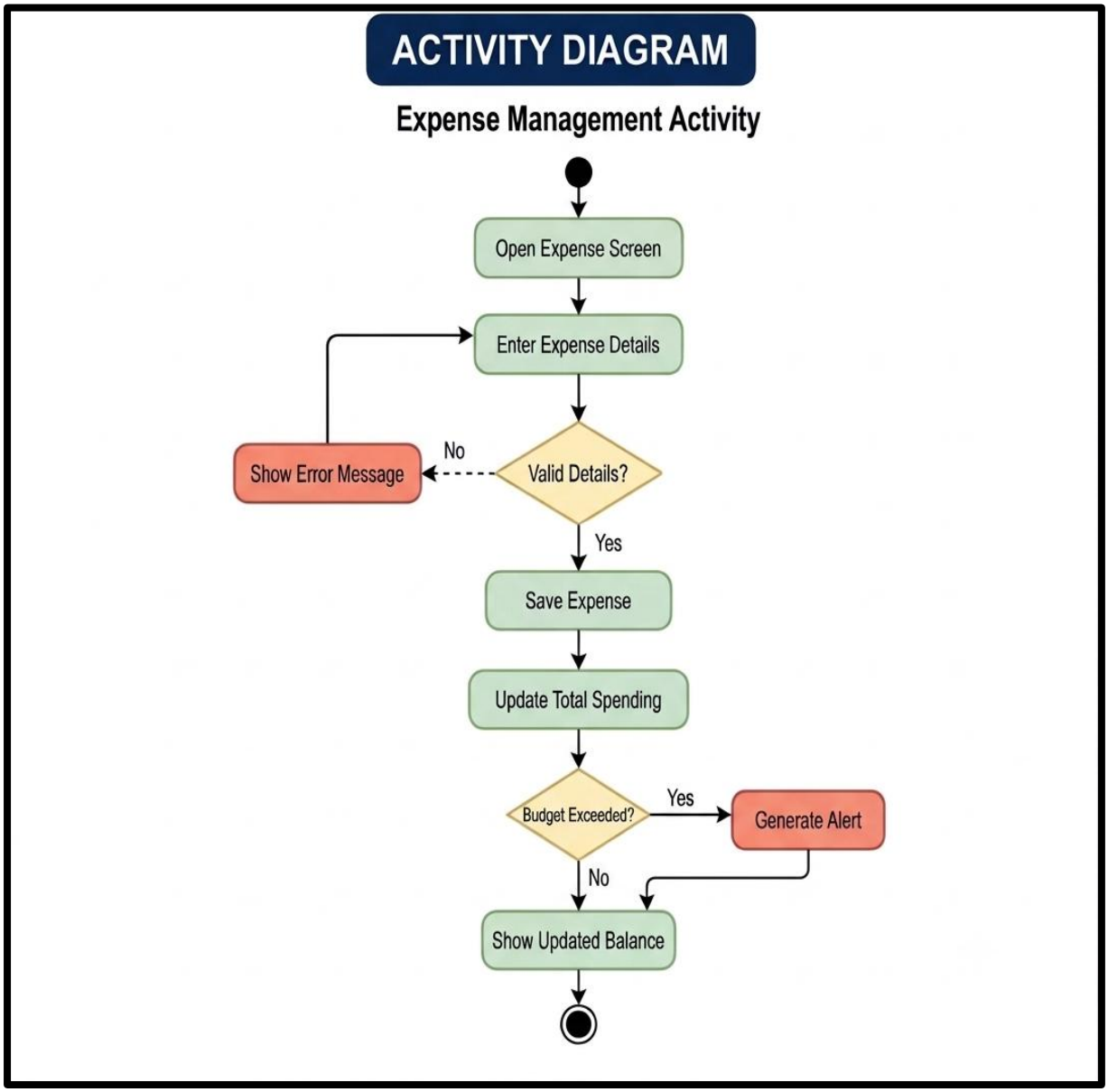


Figure 2.6: Activity Diagram of PocketPal

2.4.7 Database Design

Database design describes how the application's data is organised and stored. PocketPal uses the Room persistence library to manage structured local data.

The database configuration includes entities representing different types of information and DAO interfaces that define database operations. The application includes entity types for users, transactions, goals, and preferences. These help organise the data required by the corresponding application features.

The repository coordinates data operations between the application logic and the storage components. In addition to the Room database, PocketPal includes JSON-based local storage functionality for backup and recovery.

Using structured database entities helps organise the stored information, while DAO interfaces provide a defined way to perform supported database operations.

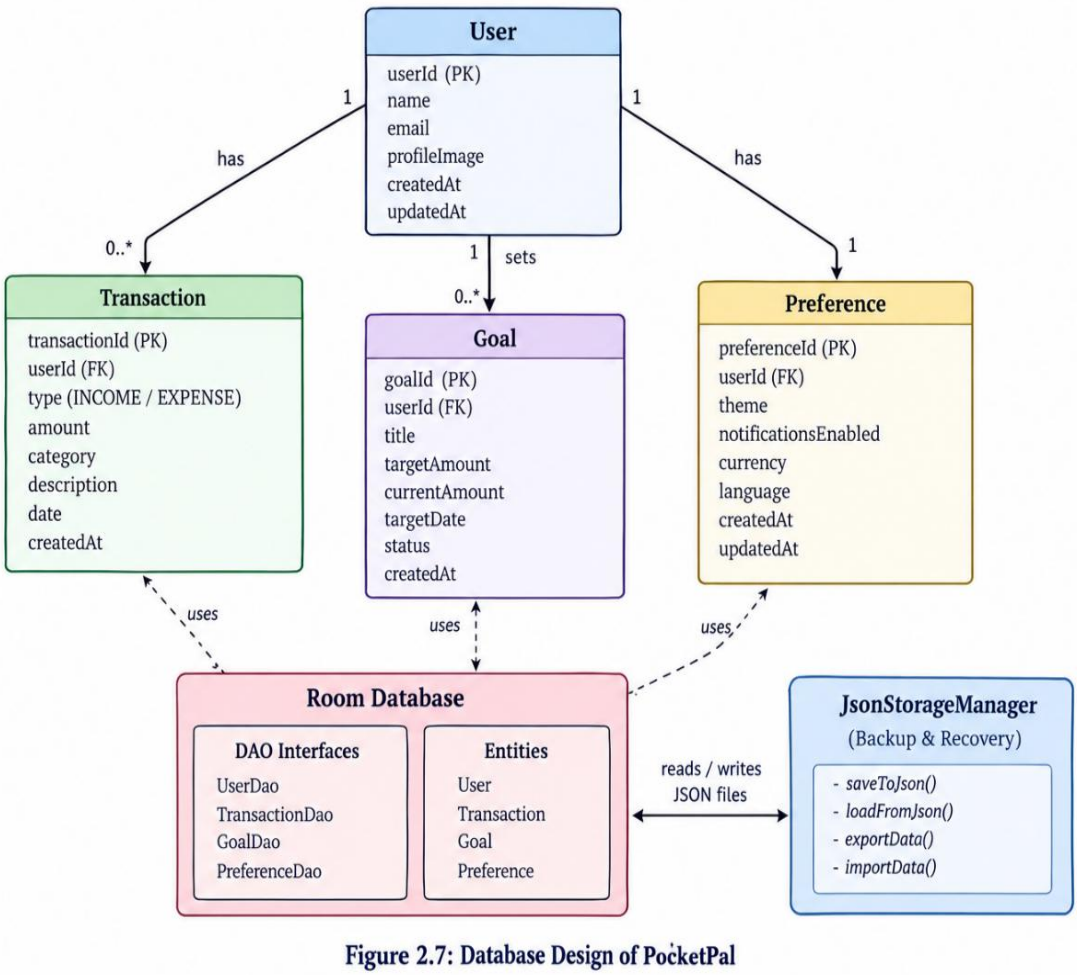


Figure 2.7: Database Design of PocketPal

2.5 Technologies Used

Several technologies and tools were used to develop PocketPal. Each technology has a specific role in implementing the application's functionality.

2.5.1 Java

Java is the main programming language used to develop PocketPal. It supports object-oriented programming and provides features such as classes, objects, inheritance, interfaces, and polymorphism.

Java was used to implement the application's logic, data models, and supporting components. It also allows related operations to be organised into separate classes, improving the readability and maintainability of the code.

2.5.2 Android Studio

Android Studio is the integrated development environment used for Android application development. It provides tools for managing project files, writing source code, configuring dependencies, building applications, and running applications on compatible devices or emulators.

It also provides debugging and development tools that help identify and correct problems during implementation.

2.5.3 Android SDK

The Android Software Development Kit (SDK) provides the tools and libraries required to develop Android applications. It includes platform APIs and other components used to build and run applications for supported Android versions.

The Android SDK supports the development of the PocketPal user interface and the use of Android application components.

2.5.4 Room Persistence Library

Room is a persistence library used to manage structured data in a local database. It provides an abstraction over SQLite and supports database access through entities, DAO interfaces, and database configuration classes.

In PocketPal, Room is used as the primary local database mechanism for structured application data. The database design includes entities for users, transactions, goals, and preferences.

2.5.5 JSON Storage

JSON stands for JavaScript Object Notation. It is a lightweight format used to represent structured data in a readable form.

PocketPal includes a JSON storage component that works with local application data for backup and recovery functionality. This provides an additional data-handling mechanism alongside the Room database.

The JSON component should not be described as a replacement for the primary database, because Room is used for the application's structured local data.

2.5.6 Gradle Build System

Gradle is a build system used to configure and build Android projects. It manages project build settings and dependencies and coordinates the steps required to compile the application.

In PocketPal, Gradle configuration files define the project's build configuration and the dependencies required for development.

2.6 Object-Oriented Programming Concepts Used

Object-oriented programming (OOP) is a programming approach in which a system is organised using classes and objects. It helps divide a program into smaller units, with each class responsible for particular data or operations.

PocketPal is developed using Java and applies several object-oriented programming concepts. These concepts help organise the application's logic, data models, and supporting components.

2.6.1 Classes and Objects

A class is a blueprint that defines the data and behaviour of a particular type of object. An object is an instance of a class created during program execution.

PocketPal uses classes to represent different parts of the application. For example, MainActivity, PocketPalViewModel, PocketPalRepository, and the transaction and goal model classes have different responsibilities. Objects created from these classes can work together to perform application operations. This makes the program easier to organise than placing all the functionality in a single class.

2.6.2 Encapsulation

Encapsulation refers to combining data and the methods that operate on that data within a class. Access modifiers can be used to control how the internal data of a class is accessed. In PocketPal, classes group related information and operations together. Where fields are declared private and accessed through suitable methods, the implementation also demonstrates data hiding.

The use of encapsulation can help reduce unwanted direct access to data and make classes easier to maintain. The exact extent of encapsulation depends on the access modifiers and methods used in each class.

2.6.3 Constructors

A constructor is a special member of a class that is called when an object is created. It is commonly used to initialise the object's data.

PocketPal contains model and supporting classes that may use constructors to initialise objects with the required information. For example, a transaction object may need values such as an amount, transaction type, category, and other transaction details, depending on the constructor provided by the class. Constructors help ensure that objects are initialised in a consistent manner.

2.6.4 Inheritance

Inheritance allows one class to acquire and extend the behaviour of another class. In Java, inheritance is implemented using the `extends` keyword. PocketPal demonstrates inheritance through Android application classes and framework classes. For example, `MainActivity` extends `ComponentActivity`, while `PocketPalViewModel` extends `ViewModel`.

These classes inherit functionality from their parent classes and provide application-specific behaviour where required. Inheritance helps the application use existing framework features instead of implementing everything from the beginning.

2.6.5 Polymorphism

Polymorphism means that a common interface or inherited method can be used with different implementations. It allows objects to respond according to their actual implementation. In Java, polymorphism can be demonstrated through method overriding and interface implementations.

In PocketPal, framework methods may be overridden in application classes, and supporting components may implement interfaces. These mechanisms allow the application to provide the behaviour required for its specific operations. The examples used in the final report should be taken from the actual source code, particularly methods that are explicitly overridden or interfaces that are implemented.

2.6.6 Abstraction and Interfaces

Abstraction means exposing the necessary operations while hiding unnecessary implementation details. It helps reduce complexity by allowing users of a class or component to work with its functionality without needing to understand every internal step. An interface in Java defines a contract that implementing classes agree to follow. It can specify methods that implementing classes must provide.

PocketPal uses DAO interfaces to define operations for accessing the local database. The application also uses supporting interfaces where required by its components. These interfaces separate the definition of an operation from the implementation that performs it. This helps organise the code and reduces direct dependence on implementation details.

2.6.7 Enums

An enum is a Java type used to represent a fixed set of named constants. It is useful when a value should be selected from a predefined group of options. PocketPal includes enums for categories and application-related values, such as transaction types, expense categories, income categories, and screen or filter options.

Using enums makes these values more organised and readable than repeatedly using unexplained text values. It also helps represent a defined set of choices consistently throughout the code.

2.6.8 Composition and Object References

Composition is a relationship in which one class contains or uses objects of other classes to perform its work. Object references allow a class to access another object and use its available methods or data.

PocketPal contains components that cooperate to perform application operations. For example, the application logic works with the repository for data-related tasks, while the repository coordinates operations involving storage components. These relationships allow the application to divide responsibilities across different classes instead of placing every operation in one class. This supports modularity and makes the code easier to maintain.

CHAPTER 3 : RESULT ANALYSIS

Splash Screen and Application Introduction

The PocketPal application begins with a welcoming splash screen designed to introduce users to the application and its purpose. The screen presents the application logo, name, and tagline, creating a simple, attractive, and user-friendly first impression.

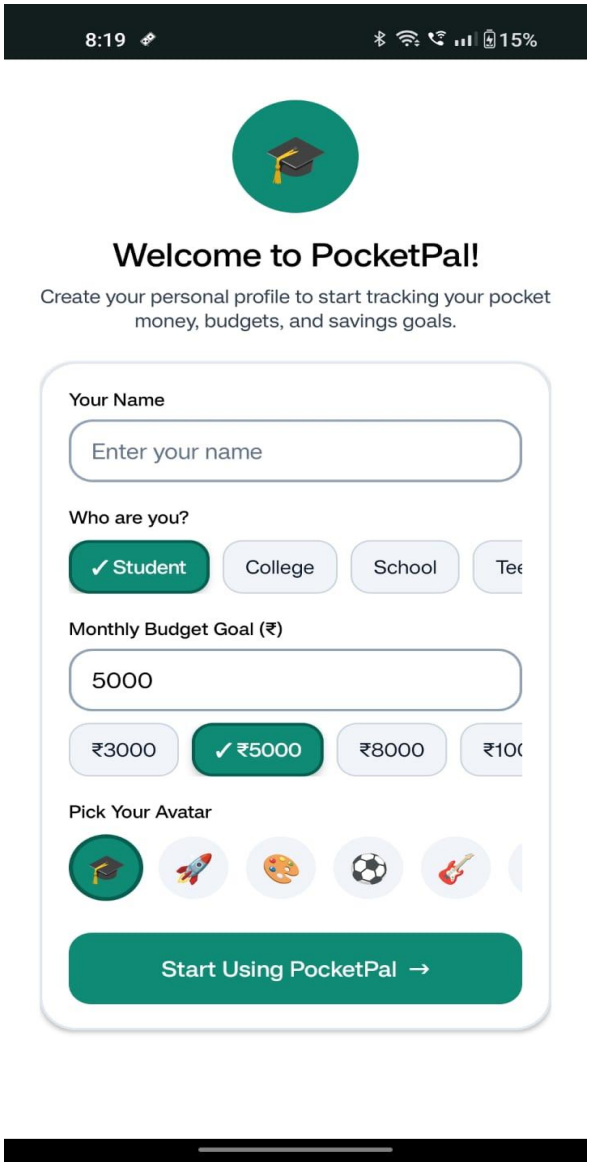
The application is designed to help users manage their personal finances by tracking income and expenses, monitoring savings goals, and developing better spending habits.

Application Name: PocketPal – Personal Expense Tracker
Tagline: Track Your Expenses, Build a Better Tomorrow
Motto: Your Money. Your Goals.



Home Screen — Dashboard

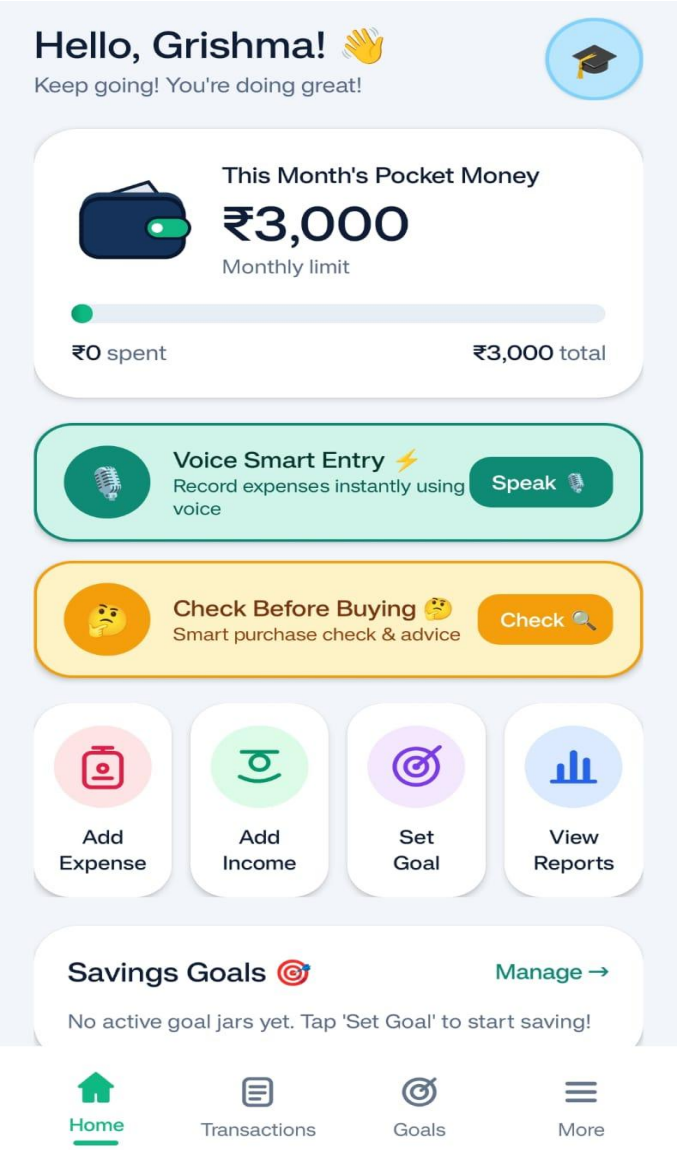
Purpose: The home screen provides an overview of the student's financial activities.



Functionalities

- **Personalized greeting:** Displays the student's name.
- **Monthly pocket money:** Shows the available budget and monthly spending.
- **Budget progress bar:** Visually represents the proportion of the budget spent.
- **Add Expense:** Opens the expense entry form.
- **Add Income:** Opens the income entry form.
- **Set Goal:** Allows the student to create a savings goal.
- **View Reports:** Opens spending reports.
- **Recent Expenses:** Displays recently recorded transactions.
- **See All:** Navigates to the complete transaction history.

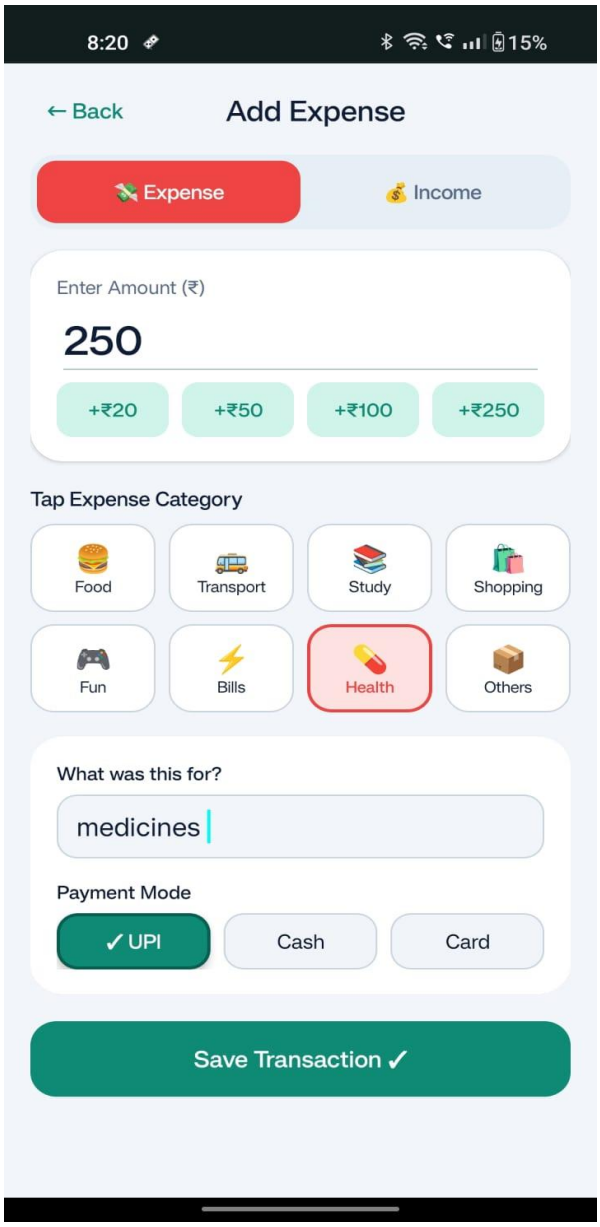
Home screen dashboard



Key features on the screen include:

- **Budget Tracking:** Highlights a monthly pocket money allowance of ₹3,000, displaying a full remaining balance of ₹3,000 with ₹0 spent.
- **Interactive Assistive Tools:** Prominently features a "Voice Smart Entry" button for hands-free expense logging and a "Check Before Buying" feature for purchasing guidance.
- **Core Action Items:** Offers quick-access buttons to record expenses, log income, define financial goals, or view analytics reports.
- **Goal Progress:** Includes a dedicated section for active savings goals, currently prompting the user to create their first goal jar.
- **App Navigation:** Provides a bottom navigation bar to toggle between Home, Transactions, Goals, and More options.

Expense Screen



Purpose

Records money spent by the student, updates financial balances, and stores data for transaction history and reporting.

Key Features & Functionalities

- **Amount Field & Quick Buttons:** Accepts the spent amount or allows rapid selection of common amounts.
- **Category Selection:** Classifies spending into 🍔 **Food**, 🚌 **Transport**, 📚 **Study**, 🛍️ **Shopping**, 🎮 **Fun**, ⚡ **Bills**, 💊 **Health**, or 📦 **Others**.
- **Description Field:** Captures notes on what the money was spent on.
- **Payment Mode:** Records the payment method (**UPI**, **Cash**, or **Card**).
- **Expense/Income Switch:** Toggles between logging an expense and logging income.

- **Save Transaction:** Validates input and saves the record.

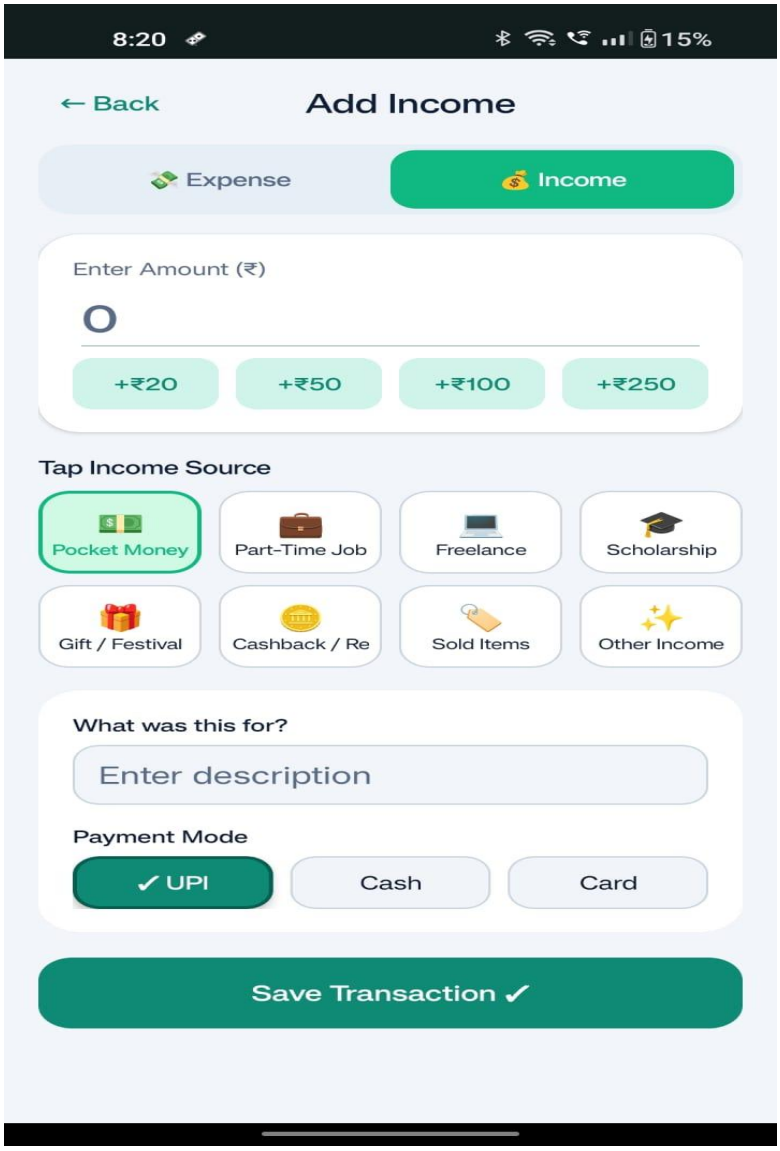
Workflow Example

1. Open **Add Expense**.
2. Enter **₹250**, select 💊 **Health**, type **"medicines"**, and pick **UPI**.
3. Tap **Save Transaction**.

System Operations & Calculations

- **Balance Update:** Updated Balance = Previous Balance − Expense Amount
- **Stored Parameters:** Transaction ID, Type, Amount, Category, Description, Payment Mode, and Date.

Income Screen



Purpose

Records money received by the student, updates financial balances, and incorporates earnings into transaction history and financial summaries.

Key Features & Functionalities

- **Amount Entry & Quick Buttons:** Accepts the incoming amount or allows fast selection via pre-set values.
- **Income Source Selection:** Categorizes income into 💵 **Pocket Money**, 💼 **Part-Time Job**, 💻 **Freelance**, 🎓 **Scholarship**, 🎁 **Gift/Festival**, 🪙 **Cashback**, 🏷️ **Sold Items**, or ✨ **Other Income**.
- **Description Field:** Captures optional notes detailing the source or reason for the income.
- **Payment Mode Selection:** Specifies the destination or form of received payment (e.g., Bank, Cash, UPI).

- **Save Transaction:** Validates and registers the entry into the system.

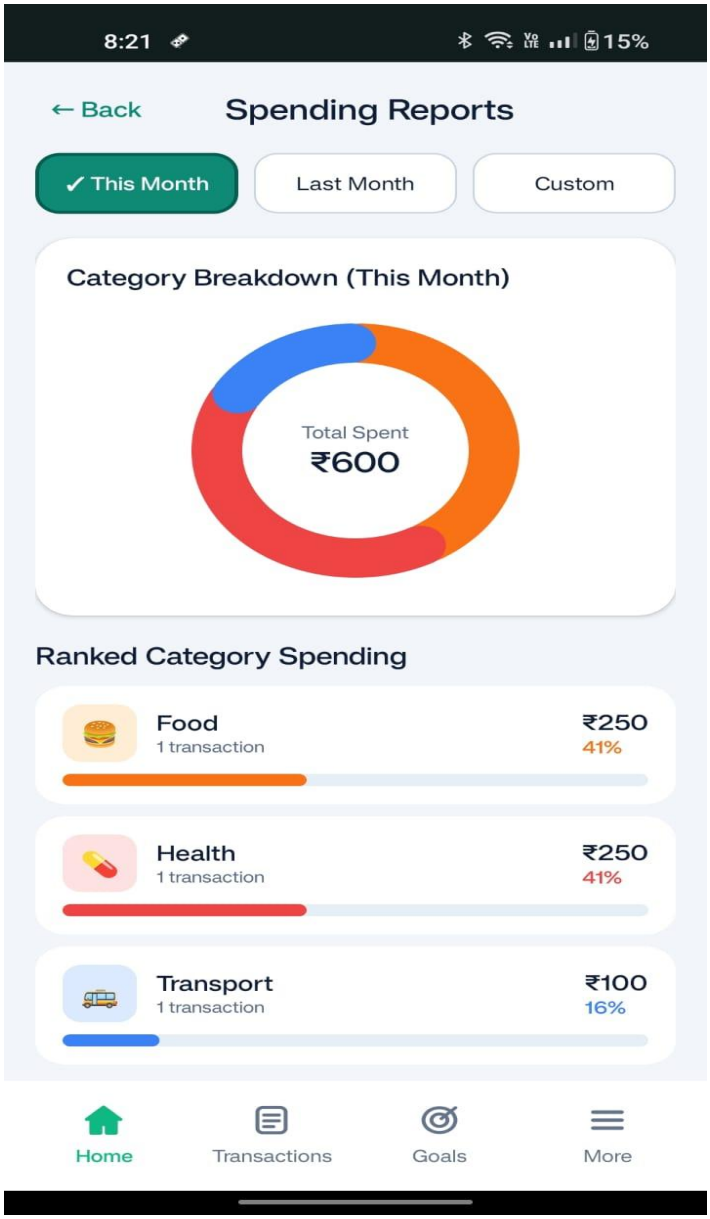
Workflow Example

1. Open **Add Income**.
2. Enter **₹2,500**.
3. Select 🎓 **Scholarship**, add a description if needed, and choose the payment mode.
4. Tap **Save Transaction**.

System Operations & Calculations

- **Balance Update:** Updated Balance = Previous Balance + Income Amount
- **Integration:** Instantly updates transaction history and recalculates budget totals and financial summaries according to system rules.

Spending Reports



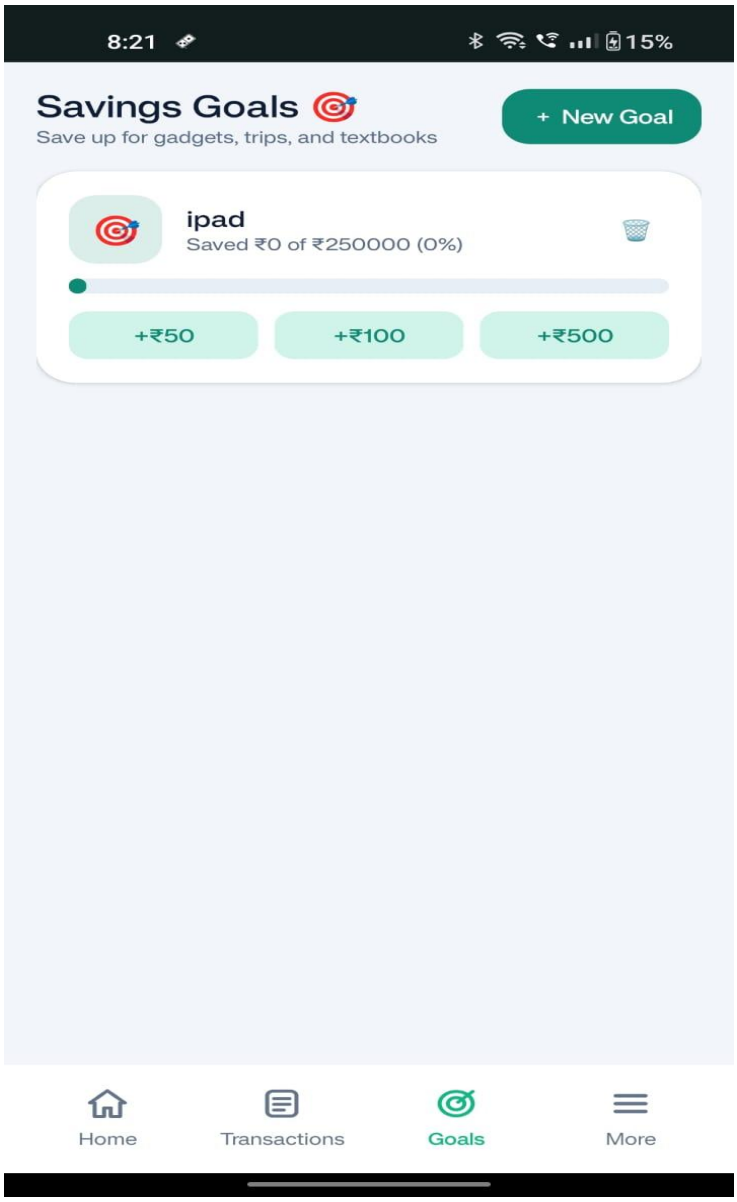
Purpose: Analyzes expenses and identifies spending patterns.

Your screenshot displays a category breakdown chart and a ranked category spending list.

Functionalities

- View the current month's spending.
- Review the previous month's spending.
- Select a custom period.
- View a category breakdown chart.
- Compare spending amounts across categories.
- See the percentage spent in each category.
- Identify categories with the highest expenditure.

Savings Goals



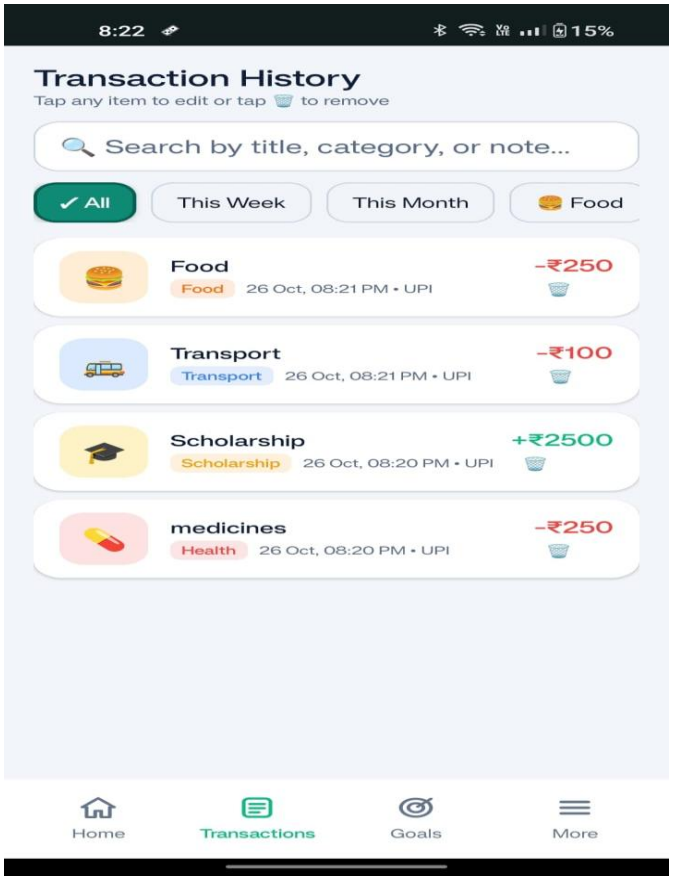
Purpose: Encourages students to save money for specific targets.

screenshot shows a goal named ipad with a target of ₹2,50,000.

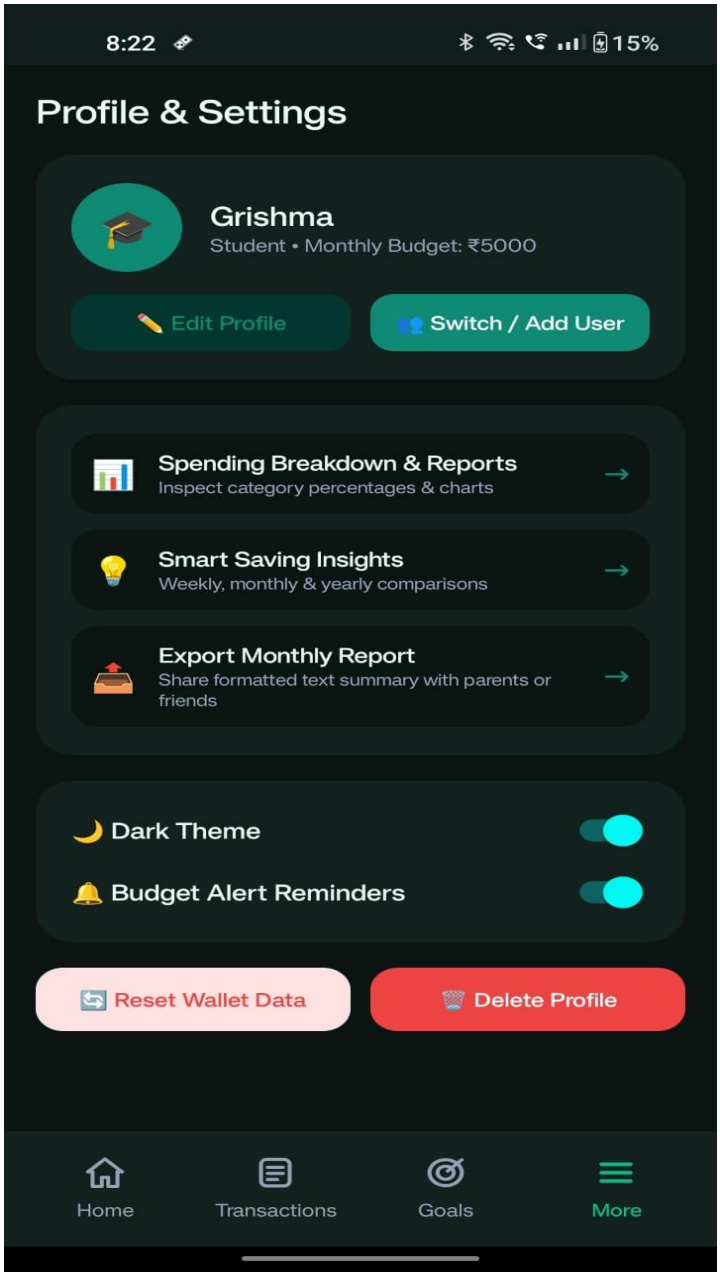
Functionalities

- Create a new savings goal.
- Specify the goal name.
- Set a target amount.
- View the amount saved.
- Monitor progress through a progress bar.
- Add money using the ₹50, ₹100, and ₹500 buttons.
- Delete a goal.

Transaction History



Profile and Settings



Purpose

Allows students to personalize application preferences, manage profile data, configure display themes, and export or reset financial data.

Key Features & Functionalities

- **Profile Management:**
  - **Edit Profile:** Updates student personal details.
  - **Switch / Add User:** Toggles between existing student profiles or creates a new one.
  - **Delete Profile:** Permanently removes the selected user profile.

- **Analytics & Reporting:**
  - **Spending Breakdown & Reports:** Launches the detailed spending reports module.
  - **Smart Saving Insights:** Opens financial insights and spending trend analyses.
  - **Export Monthly Report:** Generates and shares a downloadable monthly financial summary.
- **App Preferences & Data Control:**
  - **Dark Theme:** Toggles the UI theme between light and dark modes.
  - **Budget Alert Reminders:** Enables or disables automated budget warning notifications.
  - **Reset Wallet Data:** Clears or resets recorded wallet and financial entries.

CHAPTER 4: CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

PocketPal is an Android application developed to help students manage their pocket money in a simple and organised manner. The main purpose of the project is to help students record their income and expenses, monitor their spending, manage their budget, and set savings goals.

The application provides features such as a dashboard, expense management, income management, transaction history, spending reports, savings goals, and profile settings. These features help students maintain records of their financial activities and understand how their money is being spent. By keeping track of their expenses and savings, students can become more aware of their spending habits and make better financial decisions.

PocketPal was developed using Java and Android development tools. The project uses object-oriented programming concepts such as classes, objects, inheritance, encapsulation, and interfaces to organise the application code. The Room persistence library is used for local database management, while JSON storage provides additional backup and recovery functionality.

The development of this project provided practical experience in Java programming, Android application development, user interface design, database management, and software development. It also helped us understand how object-oriented programming concepts can be applied to develop a real-world application.

In conclusion, PocketPal provides a practical approach to student pocket money management by bringing important financial tracking features into one mobile application. It aims to encourage students to manage their money responsibly and develop better budgeting and saving habits.

4.2 Future Scope

PocketPal can be improved further by adding new features and enhancing its existing functionality. The following improvements may be considered in future versions of the application.Cloud Backup and Synchronisation: Cloud storage could be introduced to allow users to back up their financial records online and restore them when changing devices.

Advanced Spending Analysis: More detailed reports and charts could be added to help students compare their spending over different periods and identify changes in their spending habits.

Budget Notifications: Notifications could remind users when their expenses approach or exceed their set budget limits, helping them manage their spending more carefully.

Enhanced Savings Insights: The application could provide suggestions based on recorded spending patterns to help students plan their savings and work towards their financial goals.

Improved Security: Additional security features, such as application locking through a PIN or biometric authentication, could be introduced to help protect users' financial information.

User Interface Improvements: The interface could be further improved by adding more customisation options and making important information easier to access.

These improvements could make PocketPal more useful, secure, and convenient for students. They would also provide opportunities to extend the application while maintaining its main objective of helping students manage their pocket money effectively.

References

1. Android Developers. Meet Android Studio. https://developer.android.com/studio/intro
2. Oracle. Java Documentation. https://docs.oracle.com/en/java/
3. Android Developers. Android Developers Documentation. https://developer.android.com/
4. Android Developers. Build Your First App. https://developer.android.com/courses
5. Google. Material Design Guidelines. https://m3.material.io/